

Report Progetto Laboratorio di Informatica - Pipeline di Parsing Web ed Evaluation

2066423 - Lorenzo Bordi 2089078 - Claudio Frontoni 2135570 - Lorenzo Labella

1 Obiettivo del progetto

L'obiettivo del progetto è sviluppare un sistema software completo e automatizzato per scaricare, pulire e valutare testi estratti da pagine web. Nello scenario moderno, avere dati testuali puliti è un requisito fondamentale per allenare i modelli di Intelligenza Artificiale (LLM) o per alimentare i sistemi di ricerca documentale (RAG).

Il nostro programma affronta le notevoli differenze strutturali di quattro siti molto diversi tra loro: *Wikipedia*, *Il Manifesto*, *Rotten Tomatoes* e *Amazon*. Il web odierno è infatti ricco di "rumore": menu di navigazione, banner pubblicitari, pop-up, video auto-riproducenti e sezioni di raccomandazione. Il nostro estrattore rimuove in automatico tutti questi elementi superflui per restituire solo il contenuto informativo utile, formattato in un pulito standard Markdown. Oltre all'estrazione, il progetto integra una dashboard visiva e un sistema di valutazione ibrido, che unisce l'oggettività delle formule matematiche tradizionali (F1-Score) al giudizio qualitativo di un modello linguistico (LLM) operante in locale.

2 Architettura e Ingegneria del Software

L'intero ecosistema è stato "containerizzato" tramite Docker e orchestrato con Docker Compose. Questa scelta garantisce la massima portabilità: il progetto può essere avviato su qualsiasi macchina con un singolo comando, senza incorrere in conflitti di dipendenze.

2.1 Refactoring Modulare

Inizialmente concepito come un unico script monolitico, il codice Python è stato sottoposto a un profondo *refactoring*. Abbiamo suddiviso la logica in moduli con responsabilità singole (Principio *Single Responsibility*): `main.py` gestisce l'avvio, `routes.py` espone le API, `database.py` gestisce le connessioni, `parser_logic.py` contiene il mo-

tore di estrazione, `llm_judge.py` da una valutazione da parte del LLM e `utils.py` contiene funzioni utili per il backend. Questo approccio non solo rende il codice più leggibile e manutenibile, ma rispetta le migliori pratiche dell'ingegneria del software, agevolando il lavoro in team tramite Git.

3 Il Motore di Parsing (Backend)

Il cuore del sistema è sviluppato in Python utilizzando FastAPI, che garantisce prestazioni elevate grazie alla gestione asincrona delle richieste. Per lo scraping vero e proprio, ci siamo affidati a *Crawl4AI*, una libreria che pilota un browser *headless* (Chromium) capace di eseguire il codice JavaScript delle pagine, fondamentale per i siti moderni che caricano i contenuti dinamicamente.

3.1 Sfide specifiche per dominio

Ogni sito ha richiesto una strategia su misura:

- **Wikipedia:** Essendo ben strutturata, è bastato isolare l'elemento `#bodyContent` ed escludere i tag delle tabelle riassuntive e dei riferimenti bibliografici.
- **Amazon:** È risultato il dominio più ostico a causa del DOM nidificato e del forte rumore legato al marketing. Abbiamo puntato alla colonna centrale (`#centerCol`) e utilizzato filtri per rimuovere i caroselli dei "prodotti sponsorizzati" e le sezioni di up-selling.
- **Rotten Tomatoes:** A seguito di alcune criticità emerse nei primi test, abbiamo affinato la logica. Inizialmente, la rimozione delle gallerie fotografiche rischiava di "tagliare" anche la sezione fondamentale del *Cast & Crew*. Abbiamo quindi implementato una "rimozione chirurgica" tramite Espressioni Regolari (Regex), che identifica e cancella solo i blocchi indesiderati (es. "Photos", "Movie Clips",

"Critics Reviews") lasciando intatti e contigui gli attori e la trama del film.

- **Il Manifesto:** L'ostacolo principale di questo quotidiano era la presenza di *paywall* e interruzioni promozionali a metà testo. Puntando al tag `article`, il nostro parser filtra la struttura base e rimuove chirurgicamente le *call-to-action* per gli abbonamenti (es. "Registrati e scopri" o "Abbonati per 10 giorni"), restituendo la notizia pulita.

3.2 Gestione dell'HTML Fornito e Test Offline

Una delle sfide principali è stata superare il *grader automatico*, che spesso testa il sistema fornendo direttamente il codice HTML di pagine non più esistenti su internet (es. prodotti Amazon rimossi). Se il nostro parser avesse cercato l'URL online, avrebbe ottenuto un Errore 404, fallendo il test.

Per risolvere elegantemente questo problema, il backend intercetta il parametro testuale. Se l'HTML è già fornito (dal grader o dal nostro Database locale per aggirare i paywall come quello de *Il Manifesto*), il sistema sfrutta la libreria `tempfile` per creare un file temporaneo invisibile sul disco del container. A questo punto, istruiamo il crawler a leggere il file locale invece di navigare sul web, garantendo il 100% di successo nei test senza modificare la libreria di base.

4 Persistenza dei Dati (Database)

Abbiamo scelto MariaDB come sistema di archiviazione relazionale. Lo schema comprende tre tabelle strettamente collegate:

- `web_resources`: Archivia l'HTML grezzo scaricato.
- `gold_standard`: Contiene i testi di riferimento. Per garantire la compatibilità con i test automatizzati del *grader* (che richiedono un database *stateless* tra cicli di test successivi), l'inserimento utilizza il costrutto `REPLACE INTO`. Questo assicura che i dati vengano aggiornati e sovrascritti in modo pulito a ogni nuovo ciclo di valutazione, prevenendo anomalie di stato.
- `evaluations`: Una tabella di *caching* che memorizza i voti finali. Questo permette

alla dashboard statistica di calcolare le medie istantaneamente tramite query SQL, senza dover innescare nuove (e lente) inferenze dell'Intelligenza Artificiale.

5 Il Sistema di Valutazione (Evaluation)

Valutare la qualità di un testo estratto è un problema complesso, che abbiamo affrontato su due livelli complementari.

5.1 Metriche Token-Level (Ogettive)

Il primo livello è matematico. Normalizziamo sia il testo estratto (E) che il Gold Standard (G) convertendoli in minuscolo e rimuovendo la punteggiatura. Calcoliamo quindi le metriche basate sulle parole (token) in comune:

$$Precision = \frac{|E \cap G|}{|E|}, \quad Recall = \frac{|E \cap G|}{|G|}$$

Una bassa *Precision* indica che abbiamo estratto troppa "spazzatura" (come i suggerimenti di film correlati), mentre una bassa *Recall* indica che abbiamo tagliato per sbaglio parti vitali (come il Cast). Riassumiamo l'efficienza con la media armonica, l' F_1 Score:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

5.2 LLM Judge (Qualitativo)

Abbiamo usato un "Giudice AI" basato sul modello `llama3.2:3b` fatto girare localmente tramite **Ollama**. Abbiamo strutturato un *Prompt* specifico che chiede al modello di confrontare i due testi, verificando l'assenza di "allucinazioni" e la coerenza informativa. Il giudice restituisce un voto da 1 a 5 e una motivazione in linguaggio naturale, utilissima per il debug umano. Per evitare sovraccarichi termici o crash per mancanza di VRAM durante la valutazione di massa, abbiamo limitato le chiamate concorrenti a Ollama usando dei semafori asincroni (`asyncio.Semaphore`). Inoltre, per ottimizzare i tempi di inferenza su macchine prive di accelerazione hardware (CPU-bound) ed evitare problemi di *timeout*, abbiamo implementato una logica di troncamento dell'input. Entrambi i testi (estratto e gold) vengono limitati ai primi 1500 caratteri. Questa lunghezza si è rivelata il bilanciamento perfetto: riduce drasticamente il carico computazionale mantenendo sufficiente contesto semantico affinché il giudice AI possa emettere una valutazione qualitativa accurata e coerente.

6 Interfaccia e User Experience (UX)

Il frontend, sviluppato con Jinja2 e Bootstrap 5, offre un cruscotto intuitivo per interagire con il sistema. Abbiamo posto particolare cura alla *resilienza*:

- **Feedback visivi:** Durante l'attesa per il parsing o il giudizio dell'LLM, i pulsanti si disabilitano e compare uno *spinner* animato tramite JavaScript, prevenendo doppi clic accidentali.
- **Gestione Errori:** Se l'utente inserisce un dominio non supportato o Ollama è offline, l'applicazione non va in crash. Gli errori vengono catturati dal backend e mostrati all'utente tramite avvisi visivi ed eleganti.

7 Risultati e Conclusioni

Come illustrato nella Tabella 1, la pipeline dimostra capacità di astrazione eccellenti.

Dominio	Avg F1	Avg Judge
it.wikipedia.org	0.91	4.8 / 5
ilmanifesto.it	0.90	4.6 / 5
rottentomatoes.com	0.90	4.8 / 5
amazon.it	0.87	4.2 / 5
Media Finale	0.90	4.6 / 5

Tabella 1: Media delle valutazioni sui 10 URL di test.

I punteggi rasentano l'ottimo su Wikipedia, Il Manifesto e Rotten Tomatoes. Grazie all'ultimo aggiornamento delle Regex per Rotten Tomatoes, il sistema estrae il Cast senza interruzioni, portando l'F1-Score a picchi dello 0.90. Su Amazon, l'intrinseca caoticità del codice sorgente genera una lieve flessione, ma il risultato rimane ampiamente leggibile e semanticamente corretto. Il rispetto rigoroso delle direttive, unito a un codice modulare e a un'ottima UX, rende il progetto una soluzione robusta e moderna per l'estrazione dati.